18

BÖLÜM: K8S İŞ YÜKLERI & DAYANIKLILIK | MODULE: K8S WORKLOADS & SELF-HEALING RESILIENCE

Kaynak Yönetimi: Requests ve Limits (Açgözlü Pod'ları Dizginleme)

Resource Management: Requests, Limits, and OOMKilled Taming

💡 Aptala Anlatım (ELI5 Metaforu)

🇹🇷 TÜRKÇE

Açık büfe restoran kuralı: 'Masanıza oturduğunuzda en az 1 dilim böreği garanti ediyoruz (Request). Ama tabağınıza 5 dilimden fazla koyarsanız (Limit), güvenlik gelir ve tabağınızı elinizden alıp sizi kapı dışarı atar (OOMKilled)!' Kaynak sınırları koymazsanız, obur bir Spring Boot pod'u tüm sunucunun RAM'ini yiyip diğer pod'ları açlıktan öldürür!

💡 ELI5 Metaphor

🇬🇧 ENGLISH

The Golden Buffet Rule: 'When seated, we guarantee you at least 1 steak (Request). But if you attempt to hoard more than 5 steaks (Limit), security grabs you by the collar and throws you into the street (OOMKilled)!' Without limits, a gluttonous Java pod will devour the entire server's RAM, starving innocent neighboring pods to death!

🔍 Derinlemesine Teknik Bakış

🇹🇷 TEKNİK DETAY

K8s iki kaynak türünü yönetir: CPU (Sıkıştırılabilir kaynak: Limit aşılırsa süreç öldürülmez, CPU Throttling ile yavaşlatılır) ve Memory (Sıkıştırılamaz kaynak: Pod memory limitini 1 bayt bile aşarsa Linux OOM Killer devreye girer ve pod `Exit Code 137: OOMKilled` ile anında öldürülür). `requests` Scheduler'ın pod'u yerleştireceği düğümü seçerken baktığı garantili rezervasyondur. `limits` ise pod'un asla aşamayacağı tavandır.

🔍 Technical Deep-Dive

🇬🇧 TECHNICAL CORE

Kubernetes differentiates compressible and incompressible resources: CPU is throttled via CFS bandwidth control when exceeding limits; Memory cannot be throttled—exceeding limits triggers an immediate Linux kernel SIGKILL (`Exit Code 137: OOMKilled`). `requests` dictate scheduling placement; `limits` set strict cgroup enforcement ceilings.

☕ Java Spring Boot Pratiği

☕ Spring Boot in Action

```yaml
# Pod Kaynak Tanımı:
resources:
  requests:
    cpu: "200m"     # 0.2 CPU Çekirdeği garanti et
    memory: "512Mi" # 512MB RAM garanti et
  limits:
    cpu: "1000m"    # En fazla 1 tam çekirdek kullanabilir (Aşarsa yavaşlar)
    memory: "1Gi"   # 1GB'ı aşarsa K8s ACIMASIZCA öldürür (OOMKilled)!

# JVM Bellek Ayarı (Limitin %75'ini Heap yap, gerisi Metaspace/Stack kalsın):
env:
  - name: JAVA_TOOL_OPTIONS
    value: "-XX:MaxRAMPercentage=75.0 -XX:InitialRAMPercentage=50.0"
```

⚠️ Acemi Tuzağı & Pro İpucu

⚠️ Gotchas & Pro-Tips

🇹🇷 AMAN DİKKAT

JVM varsayılan olarak sunucunun toplam fiziksel belleğini görebilir! Eğer `-XX:MaxRAMPercentage` veya `-Xmx` vermezseniz, Java 32GB'lık sunucunun 8GB'ını almaya çalışır, K8s limiti olan 1GB'a çarpıp anında OOMKilled olur!

🇬🇧 WATCH OUT

Without container-aware flags (`-XX:MaxRAMPercentage=75.0`), the JVM defaults to host RAM, swiftly violating the cgroup limit and triggering instant OOMKilled panics.

🎯 Özet Çıkarımlar

🎯 Key Takeaways

- Requests zamanlama (scheduling) garantisidir; Limits ise azami tavandır.
- CPU aşımı yavaşlamaya (throttling), Bellek aşımı ani ölüme (OOMKilled) yol açar.
- Java uygulamalarında JVM bellek limitleri ile K8s limitleri tam uyumlu olmalıdır.

- Requests guarantee scheduling capacity; Limits enforce hard runtime caps.
- CPU spikes trigger throttling; Memory spikes trigger immediate OOMKilled termination.
- Always calibrate JVM MaxRAMPercentage to prevent unexpected container eviction.